# Slack App Setup

1. Creating the app

-> In the api.slack.com/apps after creating New App from Manifest, and everything from slack-app-manifest.yml as pasted there to preconfigure everything, and the app was named “Request Bot”.

-> This manifest declares the bot-user, scopes, event subscription and also the interactivity.

-> The manifest also contains the real tunnel url.

2. Declaring the BOT SCOPES

-> chat:write (lets the bot post new messages and edit its own message (chat.update)).

-> channels:history (allows it to read messages in public channels, needed for the message.channels event and also for the reconcile jobs’s conversations.history call)

-> group:history (same for private channels).

3. Installing everything to the workspace

-> After installing the bot to the Workspace, and then copying the Bot User Token in .env as SLACK_BOT_TAKEN

-> After changing the scopes the app must be reinstalled in the workspace.

4. Getting SIGNING_SECRET

-> From Basic Information -> App Credentials after getting the signing secret, we have to put it under the SLACK_BOT_TOKEN.

5. Channel and bot Membership

-> Got the channel id from Slack from the About tab then noted it in the .env as the SLACK_CHANNEL_ID.

-> We can also invite the bot in the channel by running /invite @Request Bot .

6. Event Subscriptions

-> We have to turn on Enable Events and then set the Request URL to ng-rok/slack/events.

-> Slack immediately POSTs a url_verification request with a challenge value. The app was running so it echoes the challenge back.

-> and under the “Subscribe to bot events” added the message.channels.

7. Interactivity

-> We have to turn on Interactivity and then set the Request URL to ngrok-free.dev/slack/events.

-> Without turning on Interactivity, clicking Approve or Reject does nothing for the BOT.

8. Public URL (ngrok)

-> Slack cannot reach localhost, so we have to run ngrok http 8080, which prints an address like https://abc123.ngrok-free.dev.

-> The free address changes every time ngrok restarts, so Steps 6 and 7 must be updated and saved again each time.

9. Creating the project

-> Spring Boot 4.1.1, Java 21, built with Maven. The mvnw wrapper means Maven does not need to be installed.

-> Dependencies are Spring Web, JPA/Hibernate, Flyway, the Postgres driver, validation, Lombok and the Slack Java SDK.

-> JSON uses Jackson 3 (tools.jackson), which is why the code calls asString() and not asText().

10. Database settings in application.yml

-> It points at jdbc:postgresql://localhost:5432/requests, with user and password defaulting to app/app.

-> ddl-auto: validate means Hibernate only checks the tables. Flyway creates them.

11. Slack settings in application.yml

-> slack.bot-token, slack.signing-secret and slack.channel-id each read an environment variable, with no defaults. The app will not start if one is missing.

-> .gitignore keeps .env out of git.

12. V1__init.sql

-> request holds the title, description, status (default PENDING), Slack channel and timestamp (unique together), decided_by and created/updated times.

-> slack_event_log uses the Slack event ID as its primary key, so a retried event cannot be handled twice.

13. V2__slack_message.sql

-> slack_message stores human messages from the channel (channel, timestamp, user, text), unique on channel and timestamp.

14. How migrations run

-> Flyway runs any unapplied scripts at startup and records them in flyway_schema_history.

-> Never edit an applied migration. Add a new V3__...sql instead.

15. SlackConfig

-> It builds one Slack API client from the bot token. Every class that calls Slack gets it injected.

16. Creating a request (RequestService.create)

-> It saves the row, posts a Slack message with the title, description, status and the Approve/Reject buttons, then stores the message's channel and timestamp.

-> If Slack returns an error, it throws and the database row rolls back.

17. REST endpoints (RequestController)

-> POST /requests creates a request (title required), GET /requests lists them, and GET /requests/{id} returns one or a 404.

18. Signature check (SlackSignatureVerifier)

-> It signs v0:<timestamp>:<raw body> with the signing secret and compares the result to the X-Slack-Signature header. It also rejects old timestamps. A wrong signature gets a 401.

19. Webhook endpoints (SlackWebhookController)

-> /slack/events verifies the signature, answers the url_verification handshake, and passes real events to the handler.

-> /slack/interactions reads the raw form body, because decoding it first would break the signature. It then extracts payload=.

-> Both reply 200 right away, because Slack expects an answer within 3 seconds, and the work runs in @Async methods.

20. Event handling (SlackEventHandler)

-> It inserts the event ID into slack_event_log and drops the event if that ID is already there. It ignores the bot's own messages.

-> A deleted message marks the matching request CANCELLED. Ordinary channel messages are saved to slack_message.

-> On a button click it ignores unknown or already-decided requests. Otherwise it sets APPROVED or REJECTED, records who decided, and redraws the Slack message without buttons.

21. Safety net (ReconcileJob)

-> It runs 1 minute after startup and then every 10 minutes. It checks Slack for each pending request's message and marks the request CANCELLED if the message is gone.

22. Dockerfile

-> The build stage copies pom.xml first and downloads dependencies, so they are cached between builds. It then compiles and packages the jar.

-> The run stage is a JRE-only image that runs java -jar app.jar on port 8080.

23. docker-compose.yml

-> db is Postgres 16 with a persistent volume and a health check.

-> app waits until the database is healthy, uses the host name db in its database URL, and gets the three Slack values from .env.

24. Edit feature

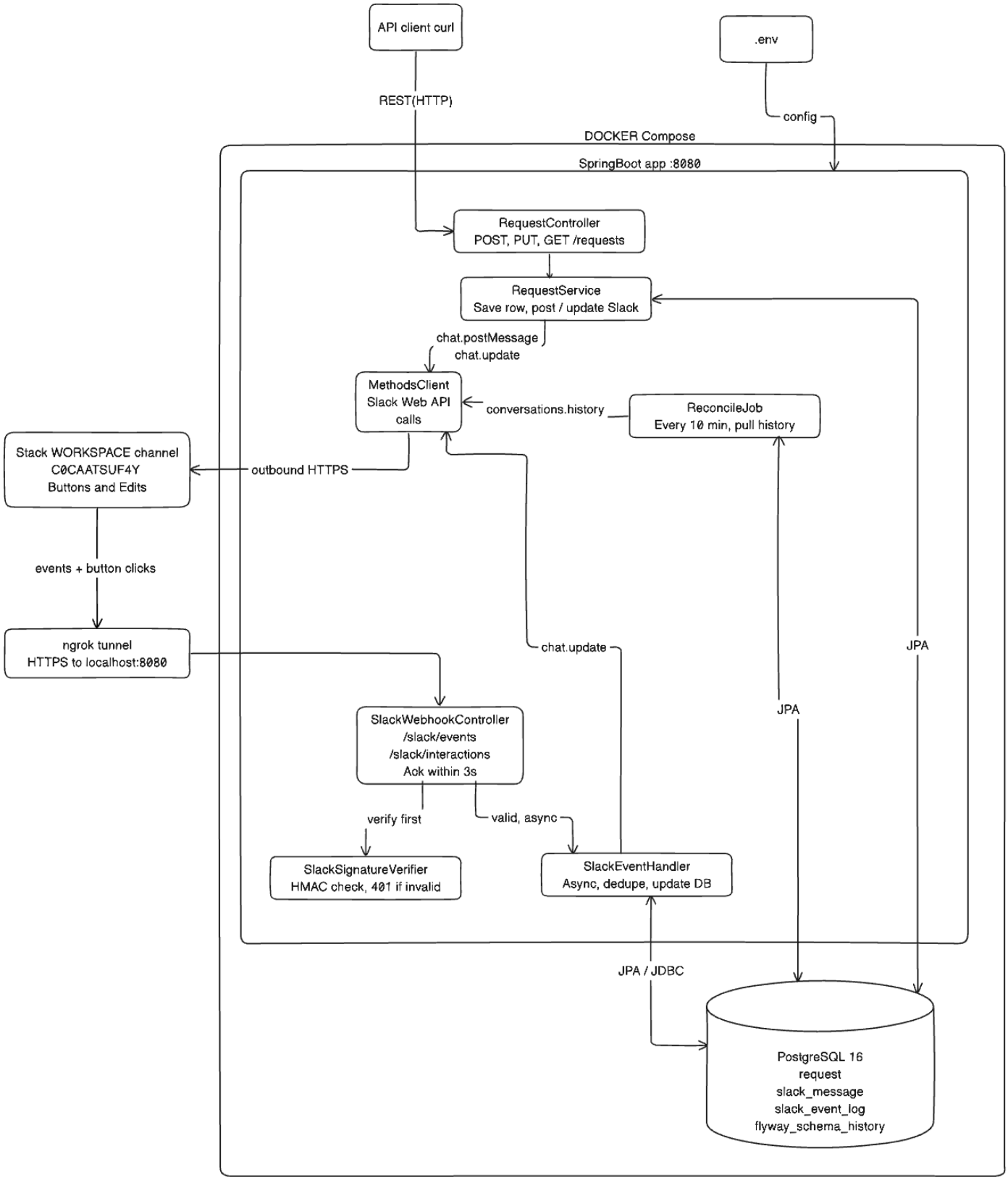
-> UpdateRequestDto has an optional title and description. A null field means unchanged.

-> RequestService.update() loads the request, applies the changes and calls chat.update, so the same Slack message changes in place. If Slack rejects it, the database change rolls back. The buttons stay only while the status is PENDING.

-> PUT /requests/{id} in RequestController is the endpoint.

-> SlackEventHandler ignores the message_changed event caused by the bot's own edit, so the app does not react to itself. When a human edits a message, it updates the saved text in slack_message.

# Slack Request Bot Architecture



# Data Flow Diagram

